

Versi 1.3
September 2026



PEMROGRAMAN WEB

MODUL 1 - FUNDAMENTAL PEMROGRAMAN WEB MODERN

Disusun Oleh:

Christian Sri Kusuma Aditya, S.Kom., M.Kom

Shofwan Alif Alfani

Nisrina Nurhafizhah

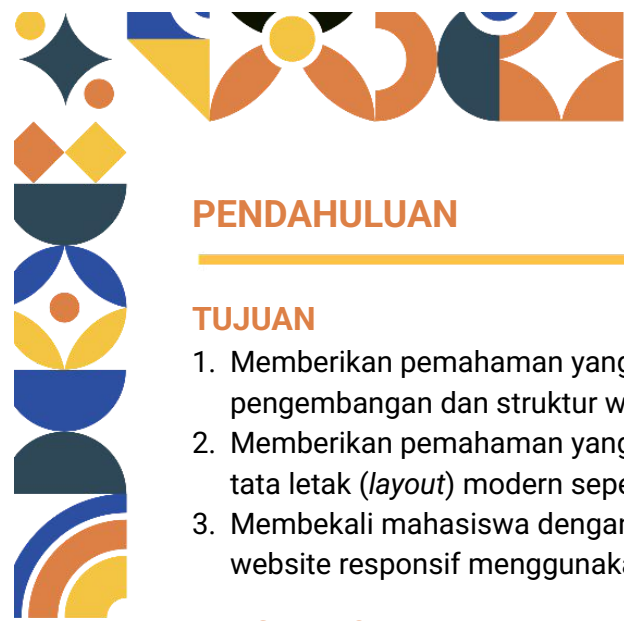
LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang



DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KEYWORDS.....	3
MATERI.....	3
1. Konsep Web.....	3
1.1. Apa itu web.....	3
2. Teknologi Fundamental Web.....	4
2.1. HTML (HyperText Markup Language).....	4
2.2. CSS (Cascading Style Sheets).....	7
• Inline CSS.....	8
• Internal CSS.....	9
• External CSS.....	10
3. Struktur Halaman Web Modern.....	11
4. Tailwind CSS.....	11
4.1. Apa itu Tailwind CSS.....	11
4.2. Cara instalasi Tailwind CSS.....	12
• Via CDN.....	12
• Via NPM (sangat direkomendasikan).....	13
4.3. Dasar penggunaan Tailwind css & membangun halaman web sederhana.....	15
• Utility Class.....	15
• Spacing (margin & padding).....	16
• Typography.....	17
• Layout.....	17
• Responsive layout.....	18
• Form dan Footer.....	18
CODELAB.....	20
Mengubah Tema Website.....	20
Melengkapi Card Section.....	20
DEMO.....	21
Studi Kasus.....	21
Ketentuan Pengerjaan.....	21
RUBRIK PENILAIAN.....	22





PENDAHULUAN

TUJUAN

1. Memberikan pemahaman yang jelas kepada mahasiswa terkait konsep dasar pengembangan dan struktur web modern
2. Memberikan pemahaman yang jelas mengenai penggunaan HTML5 semantik dan konsep tata letak (*layout*) modern seperti *Flexbox* dan *Grid*
3. Membekali mahasiswa dengan pengetahuan yang dibutuhkan untuk membangun desain website responsif menggunakan Tailwind CSS v4

TARGET MODUL

1. Mahasiswa mampu merancang desain website yang responsif menggunakan HTML5 semantik dan Tailwind CSS v4

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop atau *Personal Computer* (PC) & koneksi internet
3. *Text Editor* (Direkomendasikan menggunakan Visual Studio Code dengan ekstensi *Live Server* & *Tailwind CSS IntelliSense*)
4. Motivasi dan komitmen belajar yang kuat

KEYWORDS

Frontend, Backend, HTML5 Semantik, Box Model, Flexbox, Grid, Tailwind CSS v4



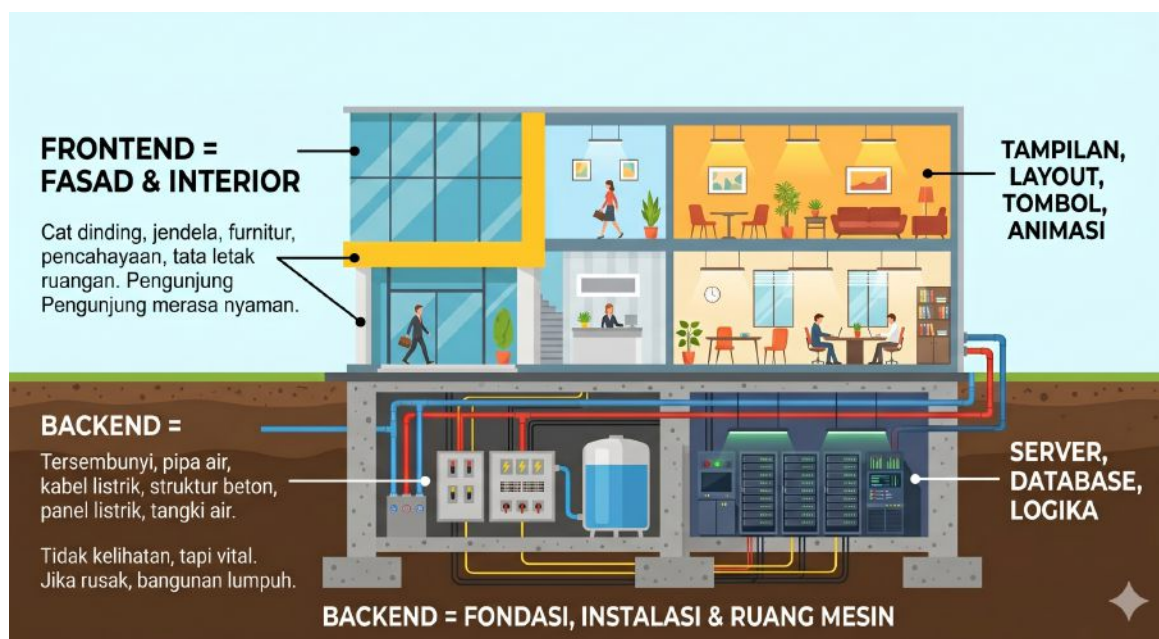


MATERI

1. Konsep Web

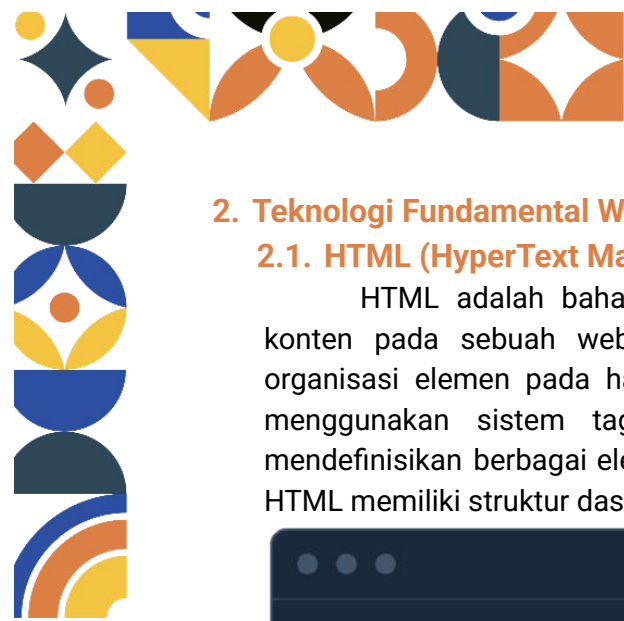
1.1. Apa itu web

World Wide Web (WWW) atau yang biasa disebut Web adalah sistem informasi yang memungkinkan dokumen dan sumber daya lainnya diakses melalui Internet menggunakan URL (Uniform Resource Locator) di browser seperti Chrome, Firefox atau Safari. Web pertama kali dikembangkan oleh Tim Berners-Lee pada tahun 1989 di CERN, Swiss, dan telah berkembang pesat menjadi fondasi utama komunikasi digital modern. Setiap website yang kita lihat di internet sebenarnya terdiri dari dua komponen utama yang saling berhubungan. Pertama adalah frontend, bagian yang tampil di layar pengguna dan berinteraksi langsung dengan pengguna. Frontend bertanggung jawab atas tampilan visual dan pengalaman pengguna (*user experience*). Kedua adalah backend, bagian yang menangani proses di balik layar dan mengatur bagaimana data diproses, disimpan, serta dikirim ke frontend. Jika dianalogikan seperti sebuah bangunan, frontend adalah seluruh bagian bangunan yang langsung terlihat dan digunakan oleh pengunjung. Mulai dari desain fasad, warna cat, tata letak ruangan, pencahayaan, papan petunjuk, hingga furnitur dan dekorasi interior. Sedangkan backend adalah bagian internal bangunan yang jarang dilihat pengunjung, tetapi sangat penting agar seluruh bangunan dapat berfungsi dengan baik. Di dalamnya terdapat fondasi beton, instalasi listrik, jaringan pipa air, ruang kontrol, panel listrik, sistem keamanan, hingga mesin pengatur air dan mesin ventilasi.



Gambar 1: Analogi website frontend & backend





2. Teknologi Fundamental Web

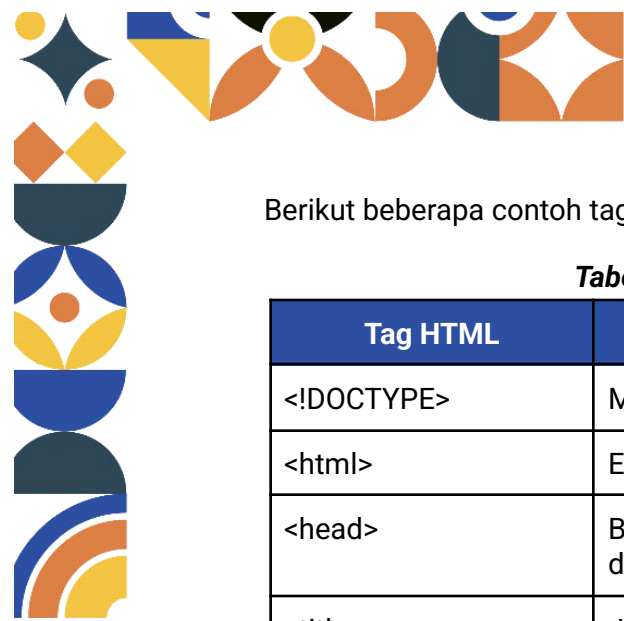
2.1. HTML (HyperText Markup Language)

HTML adalah bahasa markup yang digunakan untuk membuat struktur dasar konten pada sebuah website. Bahasa ini berfungsi untuk mengatur tampilan dan organisasi elemen pada halaman web, seperti teks, gambar, tautan, dan tabel. HTML menggunakan sistem tag yang ditulis dalam tanda kurung sudut (< >), untuk mendefinisikan berbagai elemen dengan fungsi spesifik. Secara umum, sebuah dokumen HTML memiliki struktur dasar seperti berikut:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Judul Halaman</title>
5  </head>
6  <body>
7      <h1>Hello World</h1>
8      <p>Ini adalah halaman web pertama.</p>
9  </body>
10 </html>
```

Gambar 2.1: Kode HTML sederhana





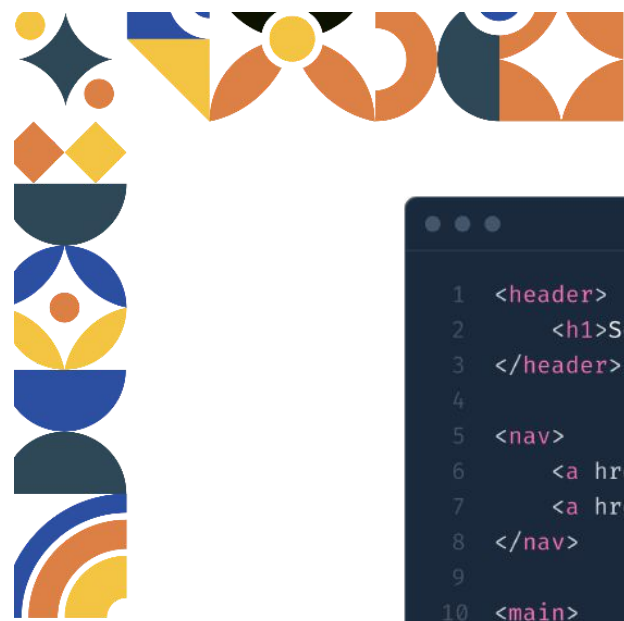
Berikut beberapa contoh tag HTML yang sering digunakan:

Tabel 2.1: Tag HTML yang sering digunakan

Tag HTML	Fungsi
<!DOCTYPE>	Mendefinisikan tipe dokumen HTML
<html>	Elemen akar (root) dari dokumen HTML
<head>	Berisi informasi meta tentang halaman, seperti judul, deskripsi, dan tautan ke file CSS
<title>	Judul halaman yang akan ditampilkan pada tab browser
<body>	Konten utama halaman yang akan ditampilkan
<h1> sampai <h6>	Heading (judul) teks dengan tingkatan variasi ukuran dari terbesar ke terkecil
<p>	Mendefinisikan sebuah teks atau paragraf
 	Mendefinisikan baris baru
<div>	Digunakan untuk menandai bagian (division) atau wadah dari sebuah struktur HTML
<table>	Membuat tabel
dan	Membuat daftar berurutan. untuk daftar, untuk item dalam daftar
	Menambahkan gambar

Seiring perkembangan teknologi web, penggunaan HTML tidak hanya berfokus pada tampilan dan struktur halaman, tetapi juga pada makna setiap elemen yang digunakan. Konsep ini dikenal sebagai **HTML semantik** (*Semantic HTML*). HTML semantik adalah penggunaan tag HTML yang memiliki arti atau tujuan yang jelas untuk konten. Sebagai contoh, dibandingkan dengan menggunakan `<div>` untuk semua bagian halaman, kita dapat menggunakan tag seperti `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, dan `<footer>` agar struktur halaman lebih mudah dipahami oleh browser, mesin pencari, maupun pengembang lain saat bekerja dalam tim. Contoh sederhana penggunaan HTML semantik:





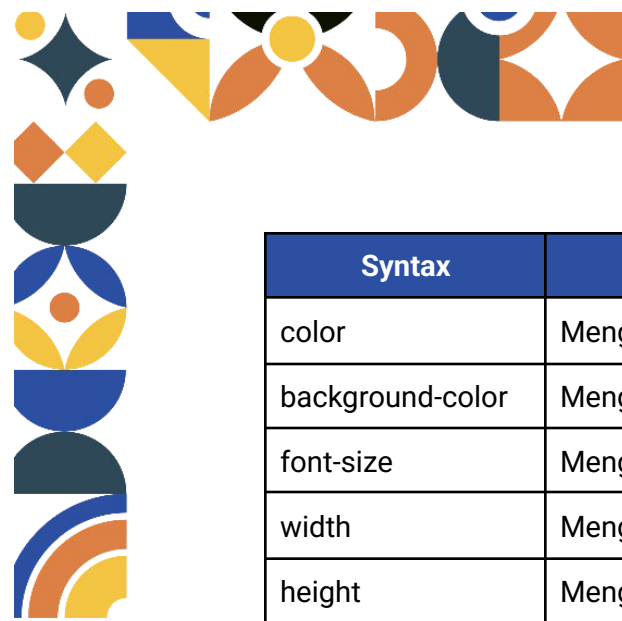
```
1 <header>
2   <h1>School Website</h1>
3 </header>
4
5 <nav>
6   <a href="#">Home</a>
7   <a href="#">Contact</a>
8 </nav>
9
10 <main>
11   <section>
12     <h2>School Profile</h2>
13     <p>Information technology based school.</p>
14   </section>
15 </main>
16
17 <footer>
18   <p>Copyright 2026</p>
19 </footer>
```

Gambar 2.2: HTML semantik

2.2. CSS (Cascading Style Sheets)

CSS (Cascading Style Sheets) adalah bahasa yang digunakan untuk mengatur tampilan, gaya, dan tata letak (layout) halaman web agar terlihat lebih menarik dan terorganisasi. CSS bekerja bersama HTML dengan cara mengendalikan bagaimana elemen-elemen HTML ditampilkan di layar, termasuk pengaturan warna, ukuran font, posisi elemen, jarak antar elemen, hingga efek visual seperti animasi atau bayangan. Dengan CSS, kita dapat mengatur properti seperti warna teks, ukuran dan jenis font, margin, padding, serta responsivitas tata letak agar halaman web tampil konsisten dan menarik di berbagai perangkat. Misalnya, CSS dapat mengubah warna latar belakang dengan properti background-color atau mengatur posisi elemen dengan properti position dan display. Dengan demikian, CSS melengkapi HTML untuk menciptakan pengalaman visual yang lebih baik bagi pengguna. Berikut ini merupakan beberapa contoh syntax CSS beserta pengertian fungsinya:





Tabel 2.2: Sintaks properti CSS

Syntax	Penjelasan	Contoh Penggunaan
color	Mengatur warna teks	color:blue;
background-color	Mengatur warna latar belakang	background-color:lightgray'
font-size	Mengatur ukuran teks	font-size: 18px;
width	Mengatur lebar sebuah elemen	Width: 300px;
height	Mengatur tinggi elemen	height: 150px;
margin	Mengatur jarak luar elemen	margin: 20px;
padding	Mengatur jarak dalam elemen	padding: 15px;
border	Memberikan garis tepi pada sebuah elemen	border: 1px solid black;
display	Mengatur tipe tata letak elemen	display: flex;
text-align	Mengatur posisi teks	text-align: center;

Agar sebuah halaman web dapat memiliki tampilan yang menarik, kita perlu menghubungkan CSS dengan struktur HTML. Terdapat 3 cara untuk memanggil CSS ke dalam HTML. Berikut ini adalah contoh cara penerapannya:

- **Inline CSS**

Inline CSS adalah penulisan CSS secara langsung pada elemen HTML menggunakan atribut style. Cara ini praktis untuk styling cepat pada elemen tunggal, namun tidak efisien untuk website yang kompleks karena membuat kode HTML menjadi berantakan dan sulit untuk di-maintenance. Contoh:

```
1 <p style="color: blue; font-size: 18px;">
2   Hello World
3 </p>
```

Gambar 2.3: Inline CSS



- **Internal CSS**

Internal CSS ditulis di dalam tag `<style>` yang biasanya diletakkan pada bagian `<head>`. Contoh:

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <style>
5          p {
6              color: green;
7              font-size: 20px;
8          }
9      </style>
10 </head>
11 <body>
12     <p>Hello World</p>
13 </body>
14 </html>

```

Gambar 2.4: Internal CSS



- **External CSS**

Cara terbaik dan paling direkomendasikan, dimana kita membuat file css terpisah dan menghubungkannya dengan HTML menggunakan tag `<link>`. Cara ini memungkinkan kita menggunakan satu file CSS untuk banyak halaman, membuat kode lebih terorganisir dan mudah di maintenance:

File HTML

```
1 <!doctype html>
2 <html>
3   <head>
4     <link rel="stylesheet" href="style.css" />
5   </head>
6   <body>
7     <p>Hello World</p>
8   </body>
9 </html>
```

Gambar 2.5: Contoh file HTML external CSS

File CSS

```
1 p {
2   color: red;
3   font-size: 22px;
4 }
```

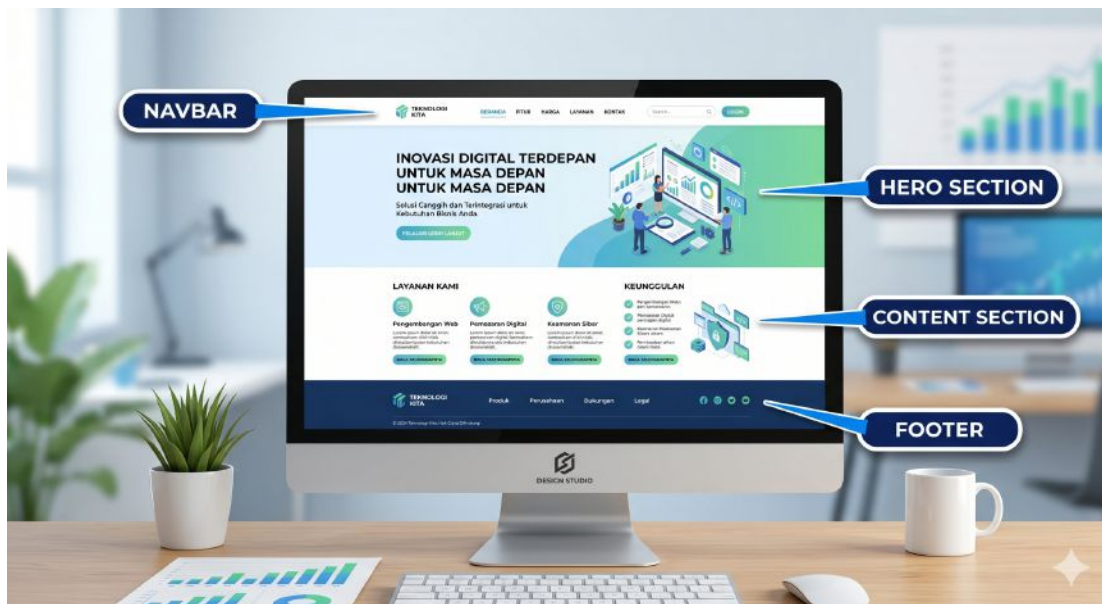
Gambar 2.6: Contoh file external CSS





3. Struktur Halaman Web Modern

Halaman web modern umumnya memiliki komponen-komponen yang tersusun secara vertikal dan terstruktur dari atas ke bawah. Masing-masing komponen memiliki tujuan dan konten yang spesifik.



Gambar 3: Contoh struktur halaman web

4. Tailwind CSS

4.1. Apa itu Tailwind CSS

Tailwind CSS adalah framework CSS berbasis utility-first yang memungkinkan kita membangun desain secara langsung di dalam HTML menggunakan kelas utility yang sudah tersedia. Berbeda dengan CSS tradisional yang biasanya mengharuskan developer membuat banyak selector dan file styling sendiri, Tailwind menyediakan class siap pakai untuk mengatur warna, ukuran, jarak, layout, typography, dan berbagai kebutuhan desain lainnya. Dengan pendekatan ini, proses pembuatan tampilan website akan lebih cepat karena styling dilakukan langsung pada elemen HTML tanpa harus menulis CSS dari awal secara berulang. Contoh sederhana penggunaan Tailwind CSS:

```
1 <button class="bg-blue-500 text-white px-4 py-2 rounded">
2   Click Me!
3 </button>
```

Gambar 4.1: Contoh sederhana penerapan CSS



Pada contoh di atas:

- `bg-blue-500` digunakan untuk memberi warna latar belakang biru
- `text-white` digunakan untuk memberi warna teks putih
- `px-4` dan `py-2` digunakan untuk mengatur padding
- `rounded` digunakan untuk membuat sudut tombol menjadi melengkung

4.2. Cara instalasi Tailwind CSS

- **Via CDN**

```

1 <!doctype html>
2 <html>
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <!-- menghubungkan tailwind css via CDN -->
7     <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
8   </head>
9   <body>
10    <h1 class="text-3xl font-bold underline">
11      Hello world!
12    </h1>
13  </body>
14 </html>

```

Gambar 4.2: Instalasi Tailwind CSS via CDN



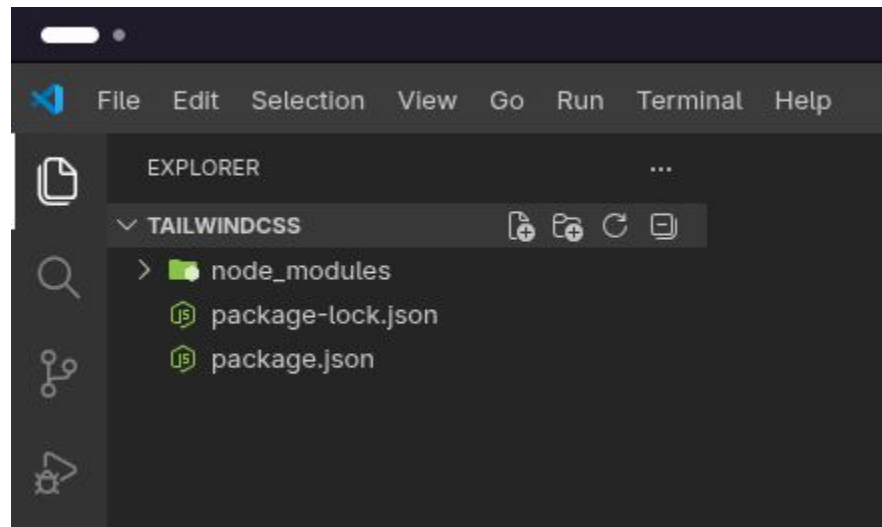
- **Via NPM (sangat direkomendasikan)**

Catatan: pastikan [Node.js](#) dan npm sudah terinstal sebelum melakukan instalasi Tailwind CSS via npm. Cek kembali modul 0 untuk cara menginstal [Node.js](#) dan npm.

Buat folder baru lalu buka folder yang baru dibuat tersebut menggunakan VS Code dan ketikkan perintah berikut di terminal VS Code:

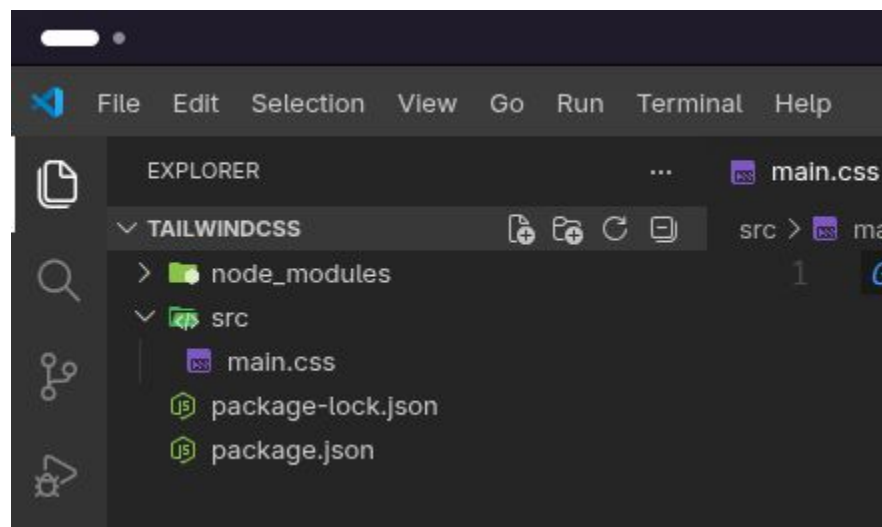
```
npm install tailwindcss @tailwindcss/cli
```

Jika berhasil, struktur folder sekarang akan menjadi seperti ini:



Gambar 4.3: Struktur folder NPM

Buat folder baru terlebih dahulu dengan nama **src** lalu buat file dengan nama **main.css** di dalam folder tersebut



Gambar 4.4: Struktur awal Tailwind CSS via NPM



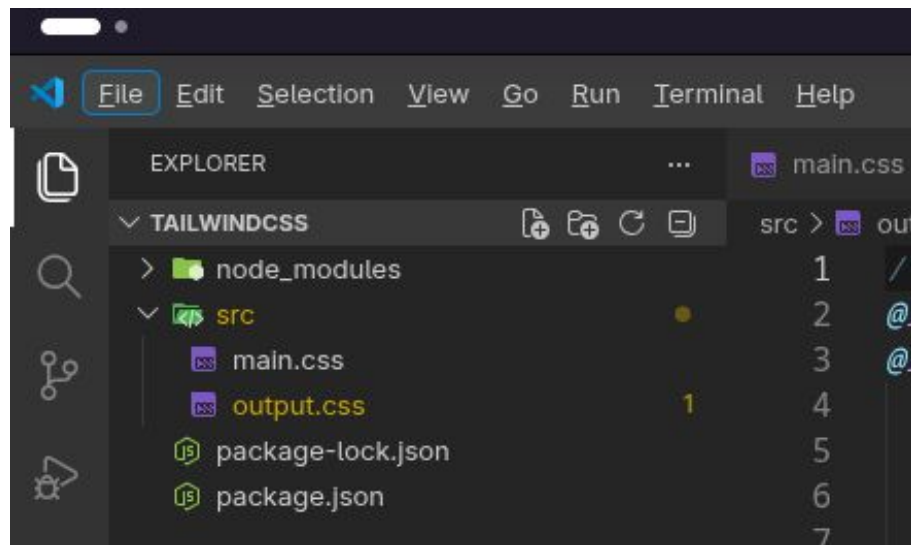
Di dalam file **main.css**, panggil Tailwind css yang sudah diinstal dengan cara menuliskan kode berikut:

```
@import "tailwindcss";
```

Kembali ke terminal, compile Tailwind cli dengan cara menuliskan perintah berikut:

```
npx @tailwindcss/cli -i ./src/main.css -o ./src/output.css --watch
```

Jika berhasil, file baru bernama **output.css** akan muncul dan berisi semua class Tailwind yang lengkap. File inilah yang nanti dipakai dan dihubungkan ke HTML



Gambar 4.5: Struktur final Tailwind CSS via NPM

Buat file **index.html** di dalam folder src dan tuliskan kode html berikut untuk mengonfirmasi apakah instalasi Tailwind css sudah berhasil:

```
1 <!doctype html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link href="./output.css" rel="stylesheet">
7 </head>
8 <body>
9   <h1 class="text-3xl font-bold underline">
10     Hello world!
11   </h1>
12 </body>
13 </html>
```

Gambar 4.6: Percobaan implementasi Tailwind CSS





Jika instalasi berhasil, output di halaman web akan menjadi seperti ini:



Gambar 4.7: Konfirmasi Tailwind CSS berhasil diimplementasikan

4.3. Dasar penggunaan Tailwind css & membangun halaman web sederhana

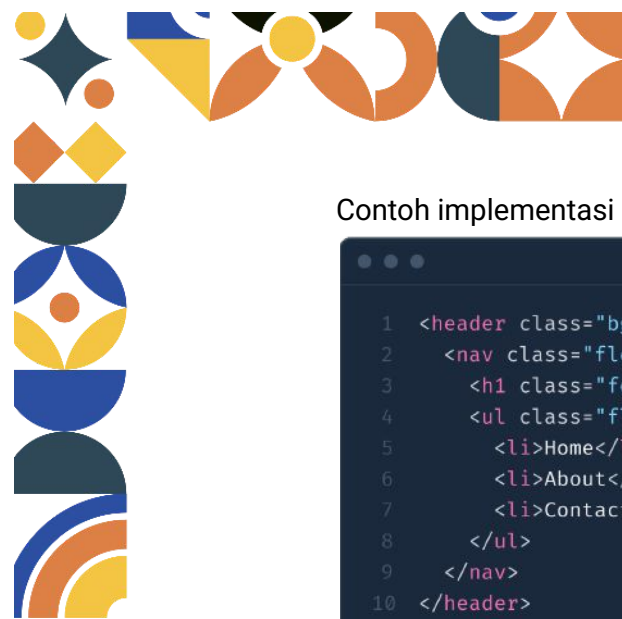
- **Utility Class**

Setiap properti CSS memiliki padanan utility class di Tailwind. Pola penamaan umumnya adalah properti-nilai:

Tabel 4: Perbandingan CSS konvensional & Tailwind CSS

CSS Konvensional	Tailwind CSS (utility)
color: white;	text-white
background-color: blue;	bg-blue-600
font-size: 1.5rem;	text-2xl
font-weight: bold;	font-bold
padding: 1rem;	p-4
margin-top: 2rem;	mt-8
border-radius: 0.5rem;	rounded-lg
display: flex;	flex
text-align: center;	text-center





Contoh implementasi membuat navbar:

```
1 <header class="bg-blue-500 text-white px-6 py-4">
2   <nav class="flex justify-between">
3     <h1 class="font-bold">MyWebsite</h1>
4     <ul class="flex gap-4">
5       <li>Home</li>
6       <li>About</li>
7       <li>Contact</li>
8     </ul>
9   </nav>
10 </header>
```

Gambar 4.8: Styling navbar dengan Tailwind CSS

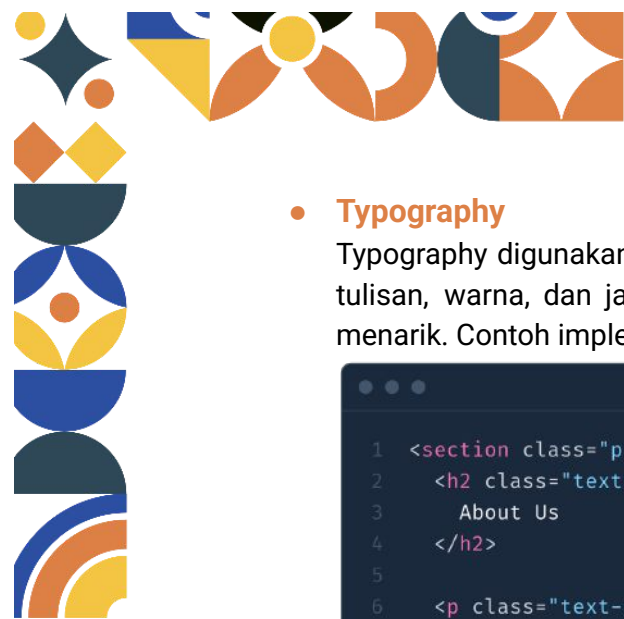
- **Spacing (margin & padding)**

Spacing digunakan untuk mengatur jarak antar elemen maupun jarak isi di dalam elemen. Pada Tailwind CSS, spacing dapat diatur menggunakan kelas seperti m-* untuk margin dan p-* untuk padding. Contoh implementasi membuat hero section:

```
1 <section class="text-center py-20 px-6">
2   <h1 class="text-4xl font-bold mb-4">
3     Learn Tailwind CSS
4   </h1>
5
6   <p class="mb-6">
7     create a website easily
8   </p>
9
10  <button class="bg-blue-500 text-white px-4 py-2 rounded">
11    Start now
12  </button>
13 </section>
```

Gambar 4.9: Spacing di Tailwind CSS





- **Typography**

Typography digunakan untuk mengatur tampilan teks, seperti ukuran font, ketebalan tulisan, warna, dan jarak antarbaris, agar konten lebih mudah dibaca dan terlihat menarik. Contoh implementasi content section:

```
1 <section class="px-6 py-10">
2   <h2 class="text-2xl font-bold text-center mb-4">
3     About Us
4   </h2>
5
6   <p class="text-gray-700 text-center leading-relaxed">
7     We provide modern web learning using Tailwind CSS.
8   </p>
9 </section>
```

Gambar 5.1: Typography di Tailwind CSS

- **Layout**

Layout digunakan untuk mengatur posisi dan susunan elemen pada halaman website. Tailwind CSS menyediakan utility seperti flex dan grid untuk mempermudah pembuatan layout modern. Contoh implementasi card layout:

```
1 <section class="grid grid-cols-3 gap-6 px-6 py-10">
2   <div class="bg-white p-4 rounded shadow">
3     Card 1
4   </div>
5
6   <div class="bg-white p-4 rounded shadow">
7     Card 2
8   </div>
9
10  <div class="bg-white p-4 rounded shadow">
11    Card 3
12  </div>
13 </section>
```

Gambar 5.2: Layouting di Tailwind CSS



- **Responsive layout**

Responsive layout digunakan agar tampilan website dapat menyesuaikan ukuran layar perangkat, baik desktop, tablet, maupun smartphone. Tailwind CSS menyediakan breakpoint seperti sm; md; dan lg: untuk mengatur tampilan responsif. Contoh implementasi pada card layout:

```

1 <section class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 px-6
  py-10">
2   <div class="bg-white p-4 rounded shadow">
3     Card 1
4   </div>
5
6   <div class="bg-white p-4 rounded shadow">
7     Card 2
8   </div>
9
10  <div class="bg-white p-4 rounded shadow">
11    Card 3
12  </div>
13 </section>

```

Gambar 5.3: Responsive layout di Tailwind CSS

- **Form dan Footer**

Form digunakan untuk menerima input dari pengguna, sedangkan footer biasanya digunakan untuk menampilkan informasi tambahan di bagian bawah website seperti copyright atau kontak. Contoh implementasi form dan footer:

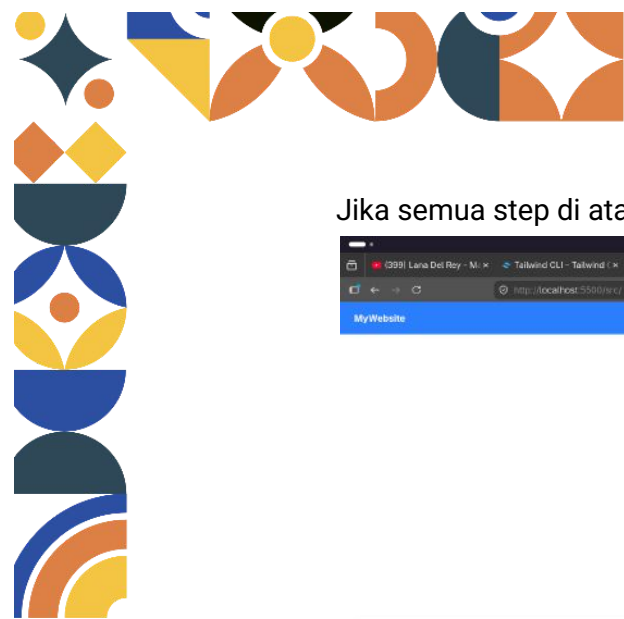
```

1 <section class="px-6 py-10">
2   <form class="bg-white p-6 rounded shadow max-w-md mx-auto">
3     <input
4       type="text"
5       placeholder="Nama"
6       class="border p-2 w-full mb-4 rounded"
7     >
8
9     <button class="bg-blue-500 text-white px-4 py-2 rounded">
10      Kirim
11    </button>
12  </form>
13 </section>
14
15 <footer class="bg-gray-800 text-white text-center py-4">
16   Copyright 2026
17 </footer>

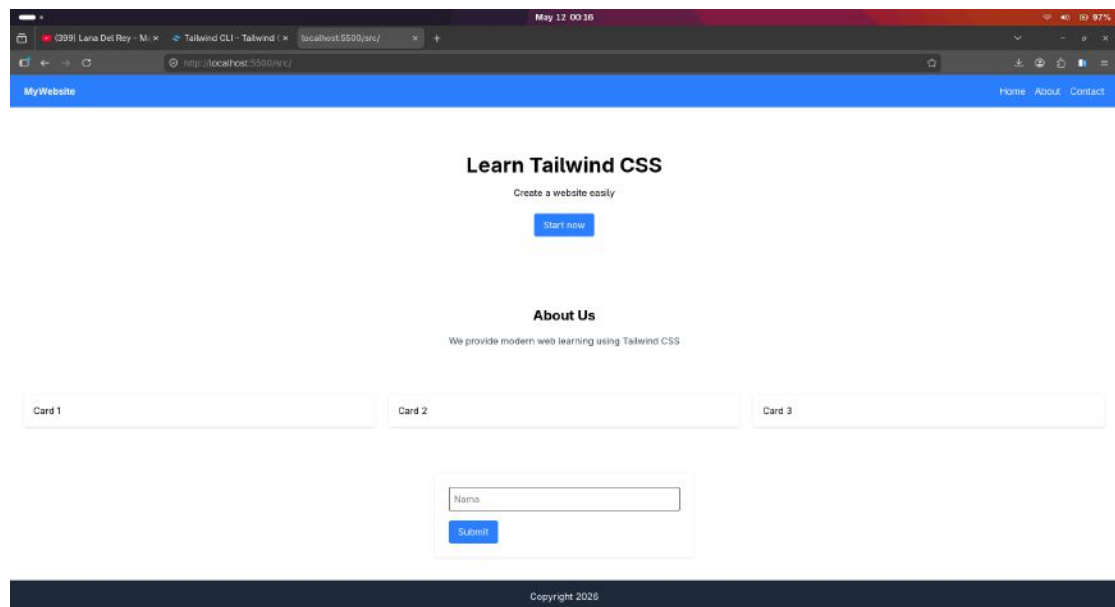
```

Gambar 5.4: Styling form dan footer tailwind css



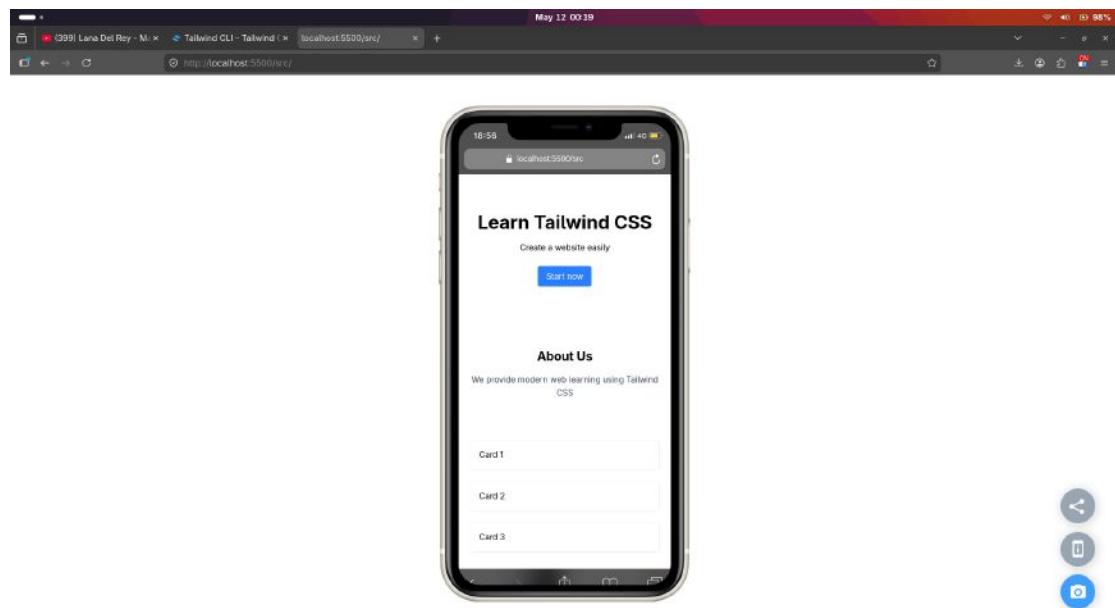


Jika semua step di atas berhasil, hasil akhirnya akan seperti ini:



Gambar 5.5: Konfirmasi web berhasil dibuat

Dan akan seperti ini jika tampil di layar mobile:



Gambar 5.6: Konfirmasi layout responsif

Selamat, Anda berhasil membangun website sederhana menggunakan HTML semantik dan Tailwind CSS 😎.





CODELAB

Mengubah Tema Website

Ubah tampilan website yang telah dibuat pada modul dengan menerapkan tema warna baru secara konsisten di seluruh halaman. Contoh tema:

- Monokrom
- Modern Blue
- Soft Pastel
- Nature Green

Clue: Gunakan konfigurasi `@theme` di file **main.css**

Melengkapi Card Section

Setiap card saat ini masih kosong. Lengkapi card dengan judul, paragraf deskripsi dan gambar. Ketentuan:

- Setiap card memiliki isi/konten yang berbeda
- Gunakan spacing dan layout yang rapi





DEMO

Studi Kasus

FlipPlay merupakan startup yang sedang mengembangkan platform hiburan digital generasi baru yang mengintegrasikan layanan streaming video dan digital reading ke dalam satu aplikasi web. Melalui FlipPlay, pengguna dapat menikmati anime, drama, manga, dan light novel dalam satu pengalaman yang terpadu. Pada tahap awal pengembangan, tim produk ingin memperkenalkan FlipPlay kepada masyarakat melalui landing page yang menarik dan informatif. Landing page ini akan menjadi fondasi awal pengembangan aplikasi yang akan terus dikembangkan sepanjang praktikum pada modul-modul berikutnya. Anda diminta untuk merancang dan membangun landing page FlipPlay menggunakan HTML semantik dan Tailwind CSS. Landing page yang dibuat harus memiliki struktur yang baik, desain yang modern, responsif pada berbagai ukuran layar, serta mampu menampilkan informasi utama platform secara jelas dan menarik.

Ketentuan Pengerjaan

- Wajib menerapkan HTML semantik
- Styling wajib menggunakan Tailwind CSS
- Struktur halaman wajib memuat minimal: navbar, hero section, content section, contact form, dan footer
- Landing page yang dibuat harus memiliki tampilan yang responsif di berbagai ukuran layar (desktop, tablet, mobile)





RUBRIK PENILAIAN

Aspek Penilaian	Poin (%)
Codelab	Total 10%
Berhasil mengubah tema	5
Berhasil melengkapi konten card	5
Demo	Total 80%
Menerapkan HTML Semantik	30
Styling menggunakan Tailwind CSS	30
Struktur minimal halaman terpenuhi	10
Responsif	10
Pemahaman	Total 10%
Ketepatan menjawab pertanyaan	10

